AROMA PACIFICO

RELEASE NOTES

Release notes and change policy

Change policy, templates and filled examples

November 2026

Every change that reaches production carries a release note. The note lets one team member approve the change with full knowledge, lets the Federation know before it happens, and tells whoever is on duty how to take it back. This document sets the rules every production change follows, and the two templates next to it give the note its shape.

Contents

1. Kinds of change
2. When a change may run
3. Approval
4. Notice to the Federation
5. Rollback
6. Where each note lives
7. What fails and what happens then
8. Assumptions and pending confirmations

Terms used

Term	Meaning in this document
Production change	Any deploy, release or rollback that touches the production environment, through the platform pipeline or an application pipeline
Visible impact	An effect a farmer, a center, a carrier, a buyer or the Federation can notice, such as a pause, a new behavior or a change in a message
Harvest week	A week that the harvest calendar of the infrastructure repository lists for a department
Harvest freeze	No production change from 05:00 to 18:00, Bogota time, on a day inside a harvest week, except in an incident
Change contact	The mailbox of the Federation that receives every notice of change
Error budget	The bad events a service level objective allows over its window of 30 days

Template	Use it for	Example
Application release note	A new version of function code through the ingestion or flight object pipeline	Delivery functions log the fields of the delivery objective
Infrastructure release note	A plan applied through the platform pipeline, such as a module version, a network, firewall, ingestion or drone path change	SMS of urgent alerts sent as transactional

1. Kinds of change

Kind	When it applies	Notice before	Window
Standard	The change has visible impact	2 working days	The change window of section 2
Low impact	The change has no visible impact, such as a log field, an alarm or an internal setting	1 working day	The change window of section 2
Incident	The change restores a service that is down or degraded, or closes a security exposure	At the start, and again at the close	Any time, including the harvest freeze

The person who writes the note picks the kind, and the approver checks it. When in doubt the change is standard. Development and test take no release note, since their pipelines deploy on each merge or tag and nobody outside the team uses them.

A rollback is a production change too. A rollback of a change that went out less than a day before reuses its note and records the reason in it. Any other rollback gets its own note.

2. When a change may run

Period	Window, Bogota time	Why
Outside a harvest week	Monday to Thursday, 08:00 to 16:00	The team is at work for the rest of the day to watch the change and take it back. A change late on a Friday could fail with nobody there over the weekend
Inside a harvest week	Monday to Thursday, 18:00 to 21:00	The harvest freeze closes 05:00 to 18:00, and the evening leaves three hours with the team still available
Friday, weekend and public holidays	No window	Same reason as Friday afternoon
Incident	Any time	A service that is down costs more than the risk of the fix

The freeze gate enforces the harvest freeze in every production job. It reads the harvest calendar after the approval and stops the job between 05:00 and 18:00 on a harvest day unless the run carries an incident reason, which stays in the run summary next to the approval. The rest of the window is a team rule that the approver checks, since a pipeline cannot know about public holidays or who is on duty.

A service level objective with no error budget left narrows the window further. Until the indicator recovers, the only production changes to that path are fixes of the cause and security patches, as the service level objectives set out.

3. Approval

Production waits for one team member to approve it in the protected `prod` environment of GitHub. The approver reads the release note and, for the platform pipeline, the plan summary of the same run, then checks four things.

1. The kind of change matches its impact.
2. The window and the notice follow sections 1 and 2.
3. The version ran successfully in test. Application pipelines also refuse a version with no successful test deployment, and the platform pipeline deploys the same commit that test ran.
4. The rollback names a trigger, the steps and what does not go back.

The approver writes the run link into the note. The person who wrote the note may approve it, since the team is small, and the approval stays in the deployment history of the environment.

4. Notice to the Federation

The notice goes by e-mail to the change contact of the Federation, in Spanish, from the sections What changes, Window and Impact of the note. It names the person on duty during the window.

Kind	Before	After
Standard	2 working days before the window	A summary the same day, with the result and anything that went back
Low impact	1 working day before	A summary the same day
Incident	When the change starts, with the cause known so far	When it closes, with the cause, the effect and what prevents it

A working day runs Monday to Friday and leaves out Colombian public holidays. When the Federation answers a notice with an objection, the change waits until the two sides agree on a new window.

5. Rollback

Every note states how the change goes back, what triggers it and what stays as it is. The two pipelines go back in different ways.

Pipeline	How it goes back	Time	What does not go back
Platform	A pull request points the environment to the previous module tag, and the same deploy runs with approval	One plan and apply, minutes for most changes	Data in tables and buckets
Ingestion and flight object	Automatic during the canary when an alarm fires. After the canary, the same workflow with the previous version moves the `live` alias back	Seconds during the canary, about 2 minutes by hand	Readings, alerts and flight records written by the new version

A change that needs both pipelines follows the order of the delivery pipelines, platform first, then code, then the removal of what the old code needed. Each step gets its own note, so each one can go back alone.

6. Where each note lives

Each production change opens an issue from the Release note template in the repository that changes, the infrastructure repository for the platform pipeline and the application repository for a function release. The issue body follows the template of this folder. The pull request and the production run link to the issue, and the issue closes once the summary has reached the Federation.

The issue history of each repository is therefore the change log of the pilot. The `CHANGELOG.md` of a module or an application lists what a version contains, and the release note says how that version reached production.

7. What fails and what happens then

Failure	How it is noticed	What happens
A change starts with no note	The approver finds no issue linked from the run	The approval waits until the note exists

Failure	How it is noticed	What happens
A standard change gets the notice of a low impact one	The approver checks the kind against the impact	The window moves until 2 working days have passed
A production job starts in the harvest freeze	Freeze gate after the approval	The job stops unless an incident reason is given
A change fails after its window, with the team gone	Alarms to the person on duty	The rollback of the note, which any team member can run from the same workflow
The change contact does not answer	No answer is needed	The change runs in its window. Only an objection stops it

8. Assumptions and pending confirmations

Assumption	If it fails	Who confirms
The Federation names one mailbox as its change contact	Each notice goes to a list of people, which the note template must record	Federation
Two working days are enough for the Federation to object to a change	The notice of a standard change grows, and fixes wait longer	Federation
The team can watch a change until 21:00 in a harvest week	The evening window shrinks, and changes wait for the end of the harvest	Team
The harvest calendar holds the weeks of each department	The freeze gate lets every hour through, and the approver checks the harvest weeks by hand	Federation

Application release note template

Copy this template into a Release note issue of the application repository for every production release of function code. Replace each text in angle brackets and keep every heading. The change policy explains the kinds of change, the windows, the approval and the notice. A filled example is in examples/application-ingestion-1.1.0.md.

```
# <Repository> <version>, <short description of the change>

| Field | Value |
|---|---|
| Kind of change | <Standard, low impact or incident> |
| Pipeline | <Ingestion or flight object> |
| Functions and versions | <Each function, its current version in production and the version this
release publishes> |
| Artifact | <Path in the artifact bucket and SHA-256 of each zip> |
| Test deployment | <Link to the successful test run of this version, with the validation hook and
smoke test result> |
| Written by | <Team member> |
| On duty during the window | <Team member> |

## What changes

<What the new version does differently, in terms a reader outside the team understands. List the
pull requests that the version contains. Say which functions keep their current version.>

## Window

| Field | Value |
|---|---|
| Date | <Day of the week and date> |
| Start and end, Bogota time | <Start> to <end> |
| Harvest week | <Yes or no, from the harvest calendar> |
| Release pace | Canary of 10 percent for 10 minutes, then all calls |

## Impact

| Field | Value |
|---|---|
| Who notices | <Farmers, centers, carriers, buyers, the Federation, or nobody> |
| Expected interruption | <None, or how long and for whom> |
| Data written differently | <New or changed fields, records or messages, and whether the previous
version can still read them> |
| Service level objectives affected | <Which objective, and how much of its error budget is left>
|
| Main risk | <What goes wrong if the version is faulty, and which alarm shows it> |

## Approval

| Field | Value |
|---|---|
| Approver | <Team member> |
| Production run | <Link to the run in the protected prod environment> |
| What the approver checked | Kind of change, window and notice, the test deployment, the rollback
|

## Notice to the Federation

| Field | Value |
|---|---|
| Sent to | Change contact of the Federation |
```

```
| Notice sent | <Date, 2 working days before for a standard change, 1 for low impact, at the start
for an incident> |
| Summary sent | <Date and result> |

## Rollback

| Field | Value |
|---|---|
| Automatic trigger | <The alarms CodeDeploy watches during the canary> |
| Manual trigger | <What the person on duty watches after the canary, and the threshold that sends
the version back> |
| Steps | Run the production release workflow with version <previous version>. It moves the `live`
alias back with the same approval |
| Time to go back | Seconds during the canary, about 2 minutes by hand |
| What does not go back | <Records, messages or files the new version already wrote, and why the
previous version can still read them> |

## Verification after the change

<The check that shows the version works in production, with the command or query and its expected
result.>
```

Infrastructure release note template

Copy this template into a Release note issue of the infrastructure repository for every production deploy of the platform pipeline. Replace each text in angle brackets and keep every heading. The change policy explains the kinds of change, the windows, the approval and the notice. A filled example is in examples/infrastructure-telemetry-ingestion-1.0.1.md.

```
# <Area> <module version or short name>, <short description of the change>

| Field | Value |
|---|---|
| Kind of change | <Standard, low impact or incident> |
| Area | <Network, center links, firewall, ingestion, drone path, security, or another root or
module> |
| Pull request | <Link to the merged pull request of the infrastructure repository> |
| Module versions | <Each module whose tag changes, from the current tag to the new one> |
| Test deployment | <Link to the deploy of test from the same commit, with the smoke test result>
|
| Written by | <Team member> |
| On duty during the window | <Team member> |

## What changes

<What the platform does differently after the apply, in terms a reader outside the team
understands. Say why the change is needed.>

| Plan of production | Value |
|---|---|
| Summary | <N to add, N to change, N to destroy> |
| Resources changed in place | <Each resource address and the attribute that changes> |
| Resources replaced or destroyed | <Each one, or none> |

## Window

| Field | Value |
|---|---|
| Date | <Day of the week and date> |
| Start and end, Bogota time | <Start> to <end> |
| Harvest week | <Yes or no, from the harvest calendar> |

## Impact

| Field | Value |
|---|---|
| Who notices | <Farmers, centers, carriers, buyers, the Federation, or nobody> |
| Expected interruption | <None, or which service stops, for how long and for whom> |
| Data | <Whether any table, bucket or stream is replaced, emptied or moved> |
| Service level objectives affected | <Which objective, and how much of its error budget is left>
|
| Main risk | <What goes wrong if the apply fails halfway, and how it shows> |

## Approval

| Field | Value |
|---|---|
| Approver | <Team member> |
| Production run | <Link to the deploy run in the protected prod environment> |
| What the approver checked | The plan summary of the same run, kind of change, window and notice,
the test deployment, the rollback |

## Notice to the Federation
```

```
| Field | Value |
|---|---|
| Sent to | Change contact of the Federation |
| Notice sent | <Date, 2 working days before for a standard change, 1 for low impact, at the start
for an incident> |
| Summary sent | <Date and result> |

## Rollback

| Field | Value |
|---|---|
| Trigger | <What the person on duty watches after the apply, and the threshold that sends the
change back> |
| Steps | A pull request points the environment to <previous module tag or previous value>, then
the deploy runs for production with approval |
| Time to go back | <One plan and apply, with the measured duration of the apply in test> |
| What does not go back | <Data written while the change was live, or a resource that a
replacement already deleted> |

## Verification after the change

<The check that shows the change works in production, with the command and its expected result,
and the next drift run that must report no difference.>
```

aromapacifico-ingestion 1.1.0, delivery functions log the fields of the delivery objective

This example fills the application template with a release of the ingestion pipeline. The version, the window, the names and the dates are an example.

Field	Value
Kind of change	Low impact
Pipeline	Ingestion
Functions and versions	Delivery status function and delivery check function, from the versions of release 1.0.0 to new versions published by this release. The ingestion function keeps its version
Artifact	`ingestion/delivery-status/1.1.0.zip` and `ingestion/delivery-check/1.1.0.zip` in the artifact bucket, each with the SHA-256 recorded in the release
Test deployment	Release 1.1.0 in test, with the validation hook and the smoke test passed
Written by	Platform team member
On duty during the window	Platform team member

What changes

The crop alert delivery objective counts live alerts handed off within 5 minutes of the reading, and it reads that from the logs of the two delivery functions. This release makes both functions write what the count needs.

- The delivery status function writes an entry named `alert_handed_off` for each push handoff, with `alert_id`, `kind`, `link` and `handoff_latency_s`, the seconds from the reading to the handoff.
- The delivery check function writes an entry named `alert_undelivered` for each alert with no handoff after 5 minutes, with `alert_id`, `kind` and `link`.

The alerts table, the messages to the farmer and the SMS rule stay as they are.

Window

Field	Value
Date	Wednesday, outside a harvest week
Start and end, Bogota time	10:00 to 11:00
Harvest week	No

Field	Value
Release pace	Canary of 10 percent for 10 minutes, then all calls

Impact

Field	Value
Who notices	Nobody outside the team. The operator gets a delivery indicator that matches the alerts table
Expected interruption	None. The `live` alias moves between two versions that both run
Data written differently	Two new log entries. The alerts table keeps the same fields, and version 1.0.0 still reads every record
Service level objectives affected	Crop alert delivery. Its indicator becomes measurable from the logs after this release
Main risk	A delivery status function that fails would stop writing handoffs, so the delivery check would report alerts as undelivered and send SMS to farmers whose push already arrived. The error alarm of the function and the undelivered alert alarm show it

Approval

Field	Value
Approver	Platform team member
Production run	Production release run of version 1.1.0 in the protected `prod` environment
What the approver checked	The change is low impact and the notice went out one working day before, release 1.1.0 passed in test, and the rollback below

Notice to the Federation

Field	Value
Sent to	Change contact of the Federation
Notice sent	Tuesday, one working day before
Summary sent	Wednesday after the window, with the first daily figure of the delivery indicator

Rollback

Field	Value
Automatic trigger	The error alarm of any of the three ingestion functions, or a message in the dead letter queue of the ingestion queue, during the canary
Manual trigger	An undelivered alert alarm for an alert that the alerts table shows as handed off, within a day of the release
Steps	Run the production release workflow with version 1.0.0. It moves the `live` alias of both functions back with the same approval
Time to go back	Seconds during the canary, about 2 minutes by hand
What does not go back	The log entries already written stay for their 30 days. The previous version ignores them, and the delivery indicator stops counting new alerts until the release goes out again

Verification after the change

The saved query of the delivery objective, run over the day after the release, returns a count of live alerts equal to the live alerts of cellular farms in the alerts table for the same day, and a count of bad alerts that matches the slow and undelivered alarms of that day.

Ingestion, telemetry ingestion 1.0.1, SMS of urgent alerts sent as transactional

This example fills the infrastructure template with a real change of the platform, the move of production from telemetry ingestion 1.0.0 to 1.0.1. The window, the names and the dates are an example.

Field	Value
Kind of change	Low impact
Area	Ingestion, alert delivery
Pull request	aromapacifico-devops-infra#27
Module versions	`terraform-aws-telemetry-ingestion` from `v1.0.0` to `v1.0.1`
Test deployment	Deploy of test from the merge commit of the pull request, with the smoke test reading found in the readings table
Written by	Platform team member
On duty during the window	Platform team member

What changes

Urgent drying alerts that reach a farmer by SMS must travel as transactional messages, the class that carriers deliver first and that marketing filters leave alone. Version 1.0.0 set that class once, on the SMS configuration set. The weekly drift check of test found that the provider leaves the class empty when it creates the set, so the setting never reached AWS and each new deploy showed the same difference.

Version 1.0.1 moves the class to each message. The delivery check function reads `SMS_MESSAGE_TYPE` with the value `TRANSACTIONAL` and passes it on every send, so the class no longer depends on the configuration set. The same version encrypts the records that wait in the archive stream with a key owned by AWS. Every archived object already landed in S3 under the objects key, and the change covers the at most 15 minutes a record waits in the stream.

Plan of production	Value
Summary	0 to add, 2 to change, 0 to destroy
Resources changed in place	`module.telemetry_ingestion.aws_lambda_function.functions["delivery_check"]` , a new environment variable. `module.telemetry_ingestion.aws_kinesis_firehose_delivery_stream.archive` , server-side encryption turned on
Resources replaced or destroyed	None

Window

Field	Value
Date	Tuesday, outside a harvest week
Start and end, Bogota time	09:00 to 10:00
Harvest week	No

Impact

Field	Value
Who notices	Nobody. A farmer receives the same SMS, now sent with the transactional class
Expected interruption	None. Lambda keeps serving the current configuration until the update finishes, and the archive stream stays active while encryption turns on
Data	No table, bucket or stream is replaced. Records in the stream during the switch keep flowing to the archive
Service level objectives affected	Crop alert delivery. Push delivery does not change, and SMS reaches only urgent alerts with no receipt after 5 minutes, which the objective does not count
Main risk	A delivery check function that fails on the new variable would stop the SMS of urgent alerts. The SMS not delivered alarm and the dead letter queue alarm of the delivery check queue show it

Approval

Field	Value
Approver	Platform team member
Production run	Deploy run of production in the protected `prod` environment
What the approver checked	The plan summary of the same run shows 2 changes in place and nothing replaced, the change is low impact and the notice went out one working day before, test ran the same commit, and the rollback below

Notice to the Federation

Field	Value
Sent to	Change contact of the Federation
Notice sent	Monday, one working day before
Summary sent	Tuesday after the window, with the result of the verification

Rollback

Field	Value
Trigger	An SMS not delivered alarm or a message in the dead letter queue of the delivery check queue within a day of the change
Steps	A pull request points the three roots back to `?ref=v1.0.0`, then the deploy runs for production with approval
Time to go back	One plan and apply of the production root, a few minutes
What does not go back	SMS already sent as transactional stay sent. Going back reopens the drift difference on the configuration set, which version 1.0.0 cannot close

Verification after the change

The next manual drift run of production reports no difference on the SMS configuration set. The configuration of the delivery check function shows the new variable.

```
aws lambda get-function-configuration --function-name <delivery check function> \
  --query 'Environment.Variables.SMS_MESSAGE_TYPE'
```

The expected answer is `"TRANSACTIONAL"`. The next urgent alert that goes to SMS shows the message type in the SMS status log.